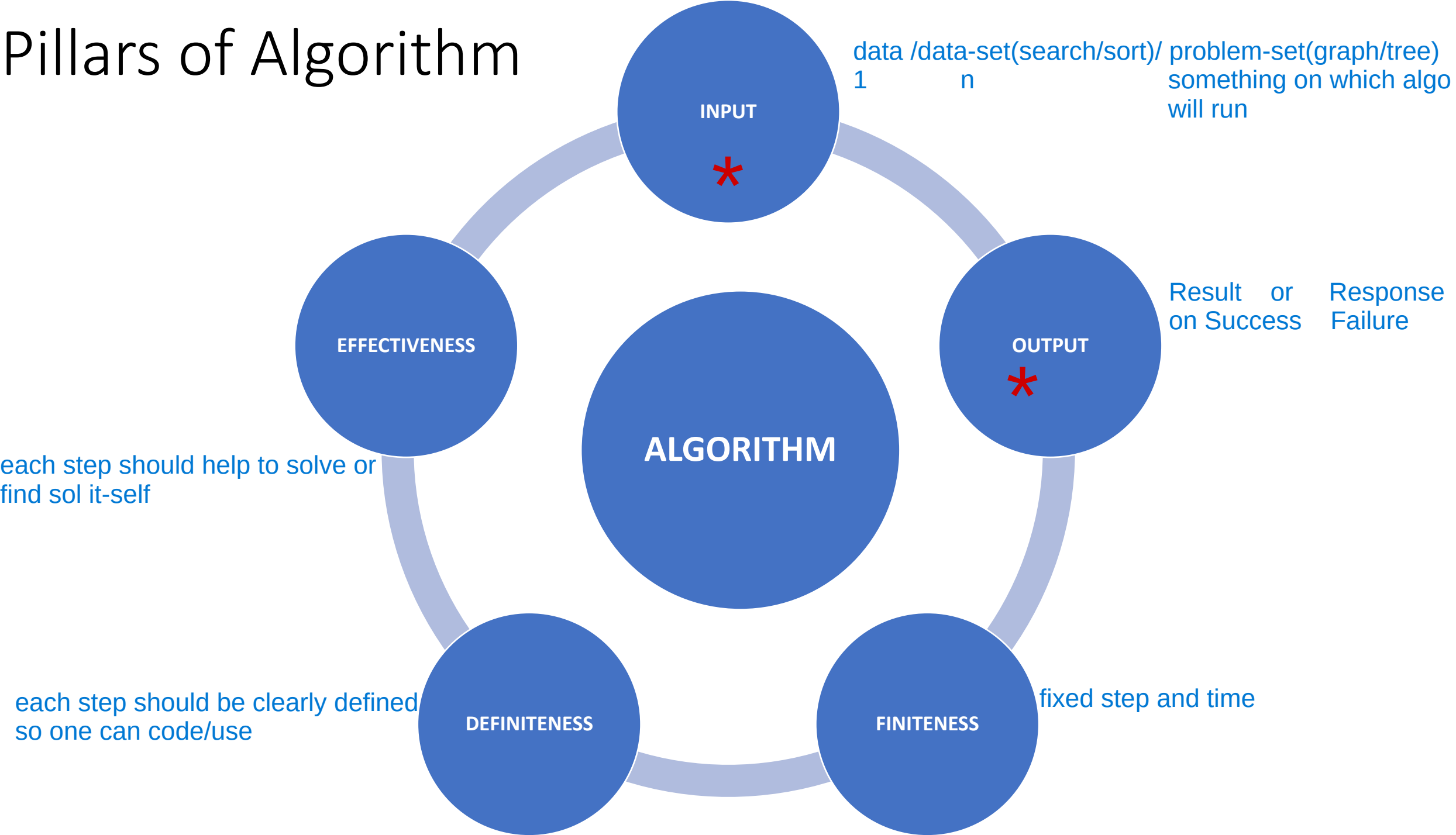


Algorithms

What is Algorithm ?

- Set of finite steps if followed will solve given problem in finite steps and finite time.

Pillars of Algorithm



Analysis ?

- **Why ?** for selection

- So one can predict time and space complexity and then select the one.

$$T(P) = t_{\text{compile}}(P) + t_{\text{execute}}(P) \quad \checkmark$$

Analysis ?

- When ?
- Priori **Before**
 - Done before implementation
 - Using pen paper **Human**
- Posteriori **After** done to recheck analysis/verify
 - Done after implementation
 - Using software by professionals **s/w and testing**

Analysis ?

- **What we get ?**
 - **Time Complexity**

- **Space Complexity**

Analysis ?

- **On What?**

- **Best Case** nearly no work hardly can say anything
- **Worst Case** max work slow but accurate
- **Average Case** nearly 50% work :fast but may/may not be accurate

} Best approach

Time Complexity

- Comments –0
- Operation----1
- Loop-----n+k

Problem

- **An algorithm takes 0.5milli seconds for 100 inputs calculate time if input is increased to 500 and if complexities are**
- **Linear**
- **Quadratic**
- **Cubic**
- **Log based**

Asymptotic Notation

Unit to record complexity using maths formula

- **Asymptotic Notation is used to describe the running time of an algorithm - how much time an algorithm takes with a given input, n.**

- **Algorithmic Common Runtimes**

- fastest to slowest are:
 - constant: $\Theta(1)$
 - logarithmic: $\Theta(\log N)$
 - linear: $\Theta(N)$
 - polynomial: $\Theta(N^2)$
 - exponential: $\Theta(2^N)$
 - factorial: $\Theta(N!)$
- Big Oh: only targets worst case ,fast but less accurate due to rules it follows
- Theta: targets AVG(best + worst) due to this slow but more accurate.
- Omega: only targets best case hence not much used

Asymptotic Notation

- **Asymptotic Notation is used to describe the running time of an algorithm - how much time an algorithm takes with a given input, n .**
- **Algorithmic Common Runtimes**
- fastest to slowest are:
- constant: $\Theta(1)$
- logarithmic: $\Theta(\log N)$
- linear: $\Theta(N)$
- polynomial: $\Theta(N^2)$
- exponential: $\Theta(2^N)$
- factorial: $\Theta(N!)$

$f(n)$:input rate--amount of input

$g(n)$:growth rate--number of steps to solve to get answer

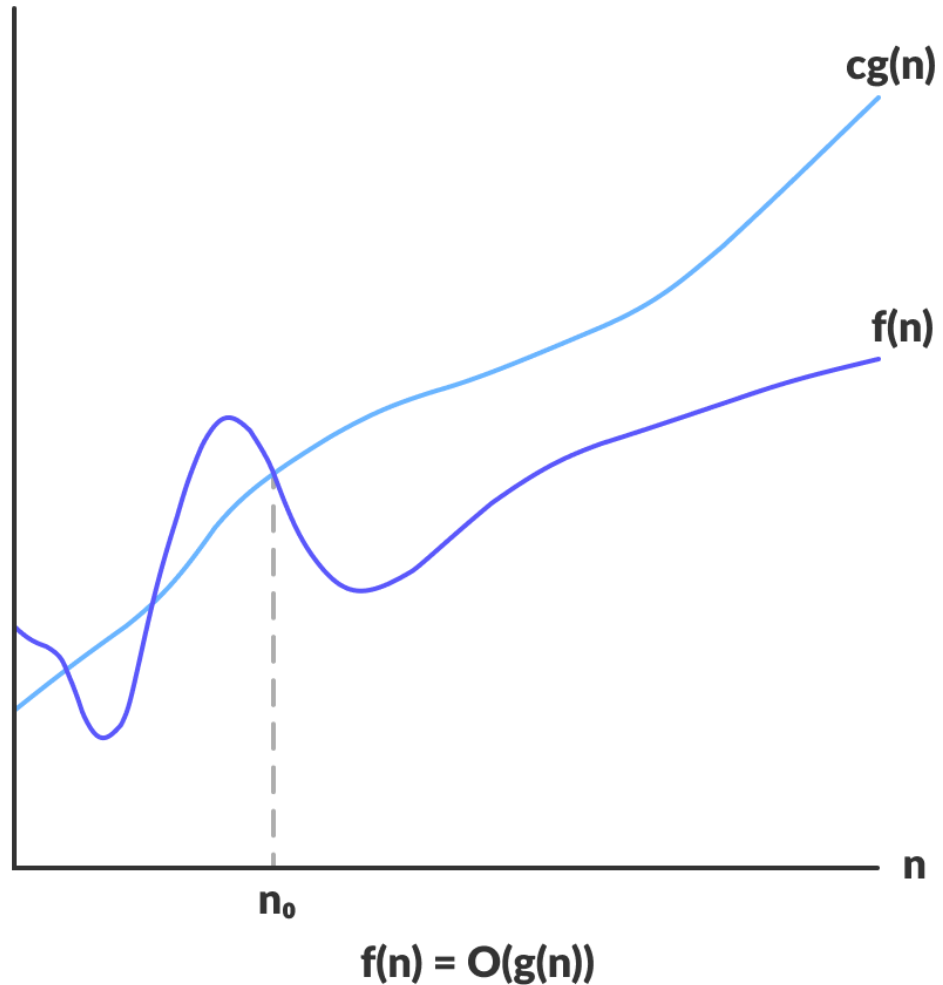
Big Oh--->Worst case $f(n) \leq c.g(n)$



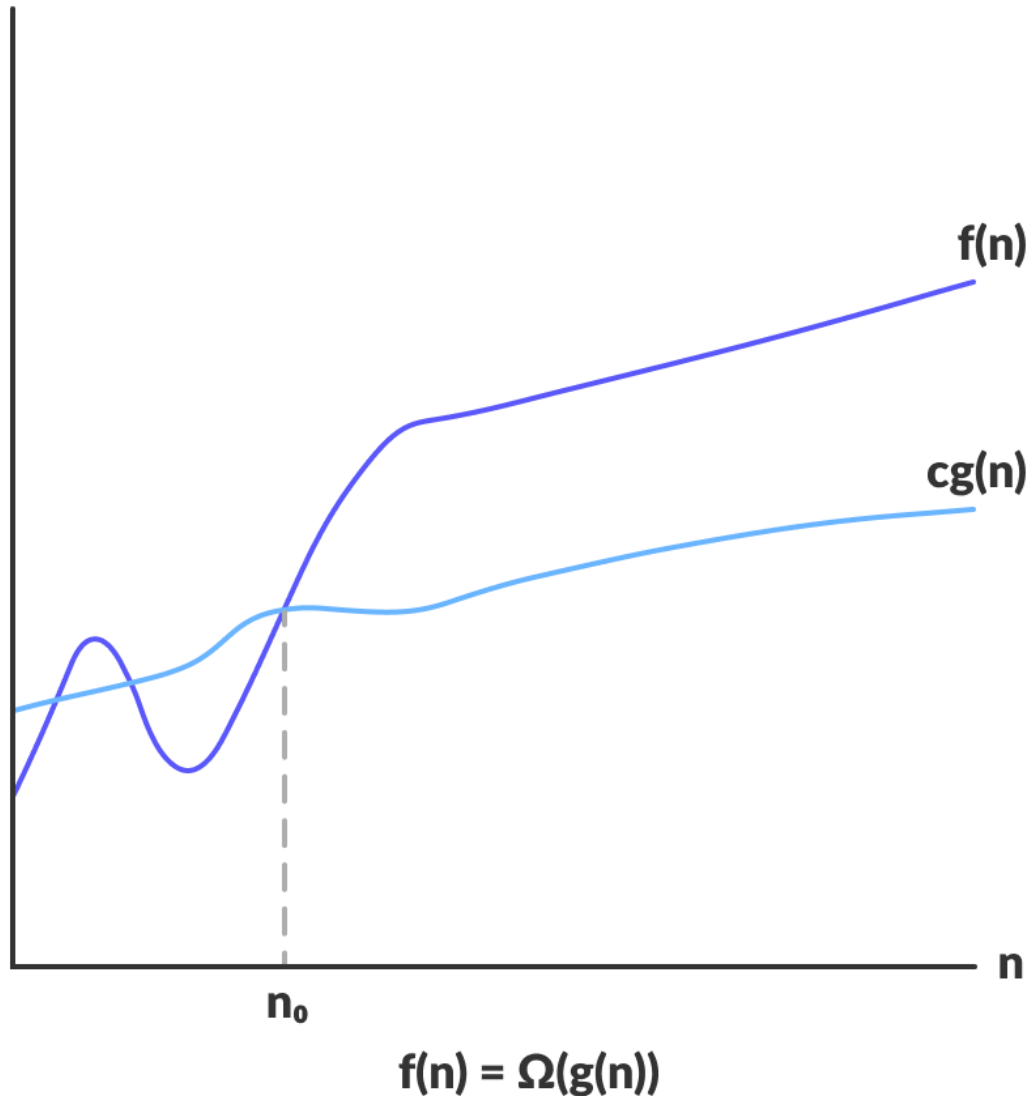
Omega--->Best $f(n) \geq c.g(n)$

Theta--->AVG(Best+Worst) $c1.g(n) \leq f(n) \leq c2.g(n)$

- **Big-O Notation (O-notation)**
- Big-O notation represents the upper bound of the running time of an algorithm. Thus, it gives the worst-case complexity of an algorithm.

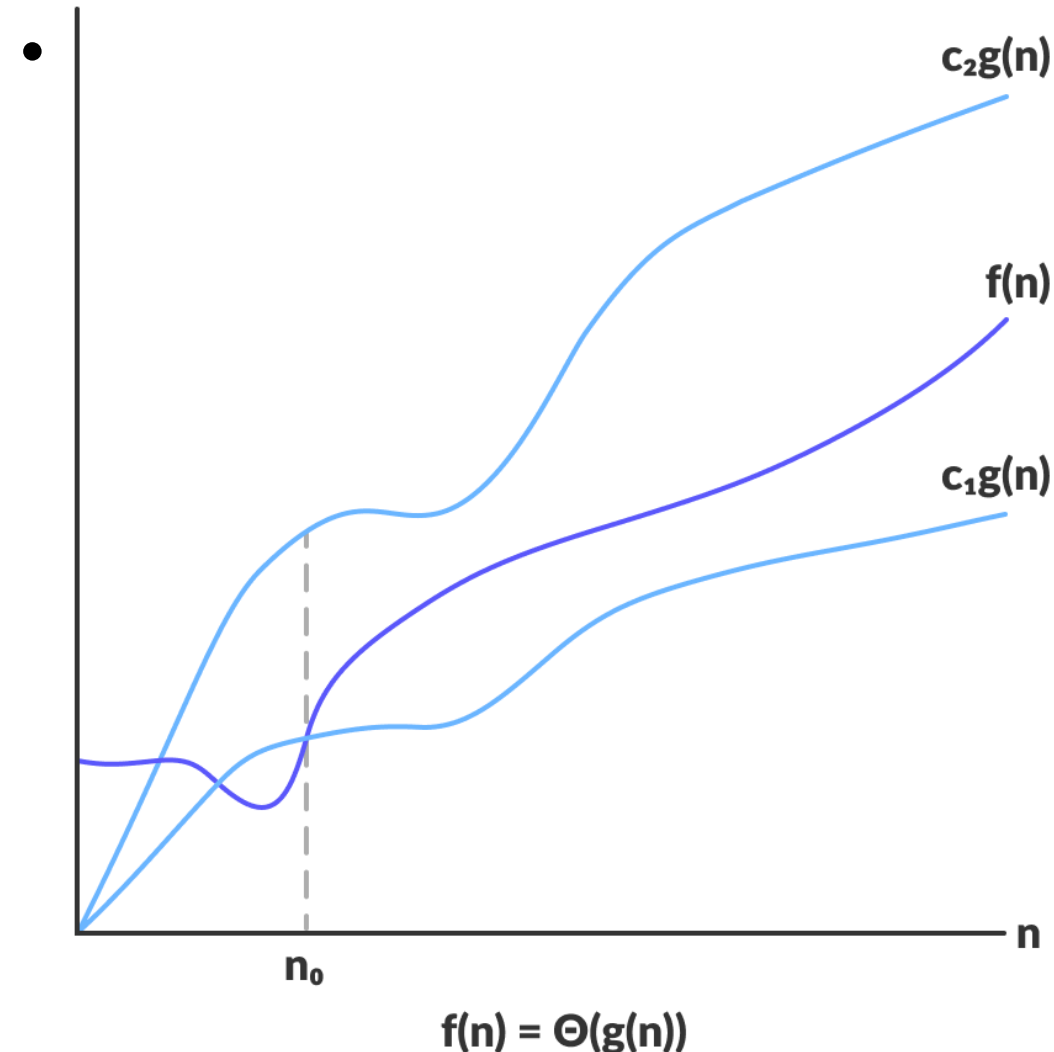


- **Omega Notation (Ω -notation)**
- Omega notation represents the lower bound of the running time of an algorithm. Thus, it provides the best case complexity of an algorithm.



- **Theta Notation (Θ -notation)**

- Theta notation encloses the function from above and below. Since it represents the upper and the lower bound of the running time of an algorithm, it is used for analyzing the average-case complexity of an algorithm.



• Big-O Notation (O-notation)

• Rules

- Constant has no value

$$O(100n) \Rightarrow O(n)$$

- In addition use max(worst only)

$$\cancel{O(n)} + O(n^3) + \cancel{O(n^2)}$$

- In nesting multiply and then use addition rule

$$O(n) \cdot O(n^2 + n)$$

$$O(n^3) + O(n^2)$$

$$O(n^3)$$

$$O(n^3)$$

Calculate Complexity using Big OH

$\sum_{i=1}^n 1$

```
void sum_of_n(int n)
{
    int sum=0;
    for(int i=1;i<=n;i++)
        sum+=(i*i);
    System.out.println(sum);
}
```

→ 1

→ 1

→ $1 + (n+1) + n$

→ 1

$O(n)$

~~$O(3n + 15)$~~

$O(3n) \rightarrow \underline{\underline{O(n)}}$